point of failure. If it's suddenly unavailable, all the applications will probably fail to start up. This risk can be easily mitigated by scaling the config server, as you would with other applications requiring high availability. When using a configuration server, it's fundamental to deploy at least two replicas.

NOTE You can use a centralized configuration server for configuration data that doesn't depend on the specific infrastructure or deployment platform, such as credentials, feature flags, URLs to third-party services, thread pools, and timeouts.

We'll use Spring Cloud Config Server to set up a centralized configuration server for the Polar Bookshop system. The project also provides a client library (Spring Cloud Config Client) that you can use to integrate Spring Boot applications with the config server.

Let's start by defining a repository for storing configuration data.

4.3.1 Using Git to store your configuration data

The config server will be responsible for providing configuration data to Spring applications. Before setting that up, though, we need a way to store and keep track of that data. Spring Cloud Config Server integrates with many different backend solutions to store configuration data. One of the most common choices is a Git repository.

First, create a new `config-repo` Git repository (for the final result, you can refer to Chapter04/04-end/config-repo). The repository can be local or remote, but for this example I recommend initializing a remote one on GitHub, just like you did for the application repositories. I'm using `main` as the default branch name.